

93 - Manage user claims

API (**A**pplication **P**rogramming **I**nterface) is a set of commands, which interfaces the programs with the processors.

Claim – a name value pair which can be used for making access control decisions. For example, we want to allow a logged-in user to be able to edit employee details, only if he has "Edit Employee" claim. We can use claims to make authorization checks like this. Since we are using claims to make authorization checks, this is called **claims-based authorization**.

Manage User Claims

Id: 7e35ba66-f855-4c61-ac1b-71c57af5c631

Email: pragim@pragimtech.com

UserName: pragim@pragimtech.com

City: London

Update Cancel

User Claims

None at the moment

Manage Claims

```
<a asp-action="ManageUserClaims"
  asp-route-userId="@Model.Id"
  class="btn btn-primary">
  Manage Claims
</a>
```

```
EditUser.cshtml X ManageUserRoles.cshtml
<div class="card-footer">
  <a asp-action="ManageUserClaims" asp-route-userId="@Model.Id"
  style="width:auto" class="btn btn-primary">
    Manage Claims
  </a>
</div>
```

Този код е достатъчен при натискане на бутон Manage Claims да бъдем пренасочени към URL

localhost:24077/Administration/ManageUserClaims?userId=87cabcf7-6319-4672-a7c8-db5a048fa5f7

пренасяйки userId чрез него, който засега връща 404, затова, нека го създадем:

```
AdministrationController.cs* X ManageUserRoles.cshtml
EmployeeManagement.Controllers.AdministrationCor ManageUserClaims(string u

[HttpGet]
0 references
public async Task<IActionResult> ManageUserClaims(string userId)
{
    var user = await userManager.FindByIdAsync(userId);

    if (user == null)
    {
        ViewBag.ErrorMessage = $"User with Id = {userId} cannot be found";
        return View("NotFound");
    }

    return View(user);
}
```

Сложихме кода, ако не е намерен потребител с такова ID. За да продължим, трябва да извлечем списък с всички claims за нашето приложение. Те могат да дойдат от:

- * таблица БД;
- * конфигурационен файл;
- * статичен клас.

Нека опитаме в статичен клас Models folder -> Add new class -> ClaimsStore.cs

```
ClaimsStore.cs* X AdministrationController.cs* ManageUserRoles.cshtml
EmployeeManagement.Models.ClaimsStore

using System.Collections.Generic;

namespace EmployeeManagement.Models
{
    0 references
    public static class ClaimsStore
    {
        public static List<Claim>

        using System.Security.Claims;
        The name 'Claim' does not exist in the current context
        using System.Collections.Generic;
        using System.Security.Claims;
    }
}
```

Изпуснала съм интервал на Create Role

```
public static List<Claim> AllClaims = new List<Claim>()
{
    new Claim("CreateRole", "CreateRole")
}

2 of 7 Claim(string type, string value)
overloaded Initializes a new instance of the Claim class with the specified claim type, and value.
version value: The claim value.
```

```
using System.Collections.Generic;
using System.Security.Claims;

namespace EmployeeManagement.Models
```

```

{
    public static class ClaimsStore
    {
        public static List<Claim> AllClaims = new List<Claim>()
        {
            new Claim("Create Role", "Create Role"),
            new Claim("Edit Role", "Edit Role"),
            new Claim("Delete Role", "Delete Role")
        };
    }
}

```

UserClaimsViewModel Class

Извеждаме списъка с всички Claims, като използвайки foreach loop минаваме през всички и ако дадено право се повтаря, маркираме check-box-a му, ако не, оставяме го празно.

UserClaimsViewModel Class

```

public class UserClaimsViewModel
{
    ctor public UserClaimsViewModel()
    {
        Claims = new List<UserClaim>();
    }

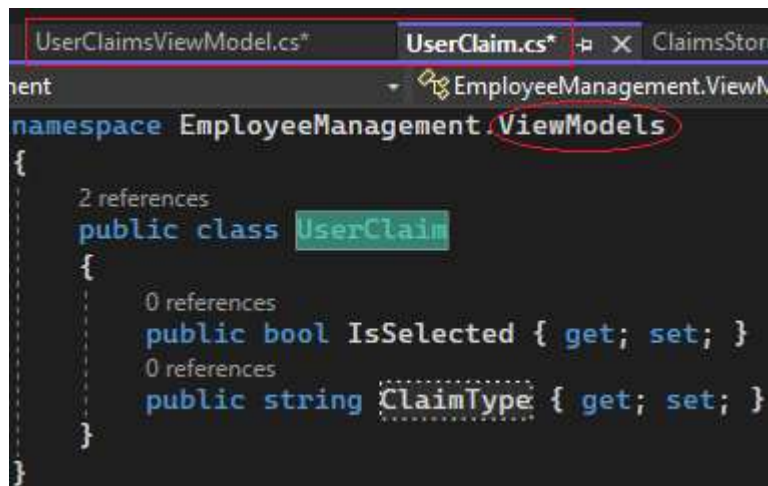
    1 public string UserId { get; set; }
    n public List<UserClaim> Claims { get; set; }
}

public class UserClaim
{
    public string ClaimType { get; set; }
    public bool IsSelected { get; set; }
}

```

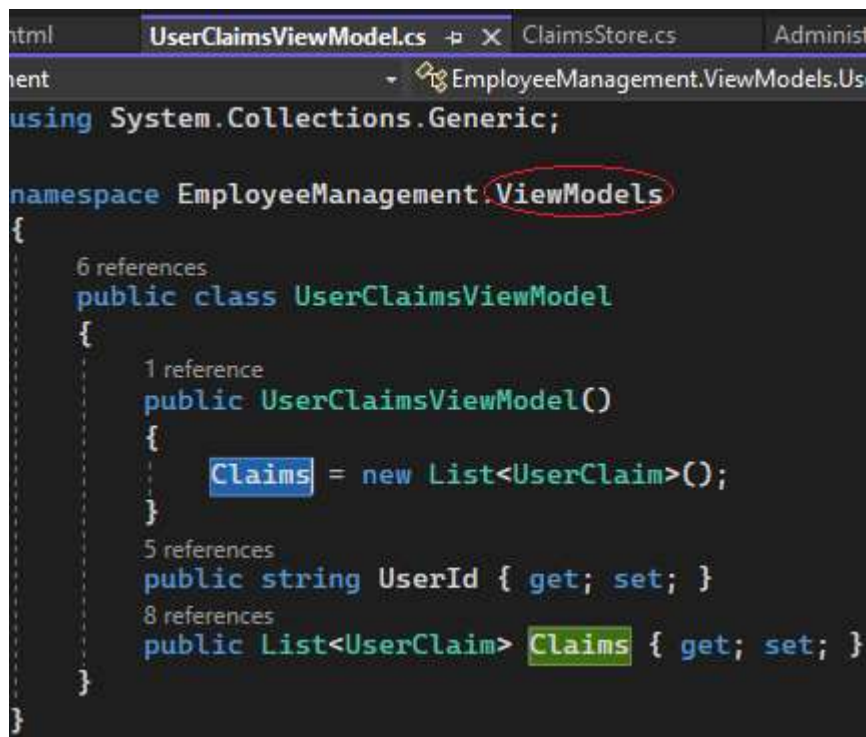
user : claims
1 : n

Използваме допълнителния конструктор UserClaimsViewModel, въпреки, че може без него (директно с UserId + Claims), за да инициализираме първоначални данни за Claims, избягвайки null reference exceptions.



```
namespace EmployeeManagement.ViewModels
{
    2 references
    public class UserClaim
    {
        0 references
        public bool IsSelected { get; set; }
        0 references
        public string ClaimType { get; set; }
    }
}
```

```
namespace EmployeeManagement.ViewModels
{
    public class UserClaim
    {
        public bool IsSelected { get; set; }
        public string ClaimType { get; set; }
    }
}
```



```
using System.Collections.Generic;

namespace EmployeeManagement.ViewModels
{
    6 references
    public class UserClaimsViewModel
    {
        1 reference
        public UserClaimsViewModel()
        {
            Claims = new List<UserClaim>();
        }
        5 references
        public string UserId { get; set; }
        8 references
        public List<UserClaim> Claims { get; set; }
    }
}
```

```
using System.Collections.Generic;

namespace EmployeeManagement.ViewModels
{
    public class UserClaimsViewModel
    {
        public UserClaimsViewModel()
        {
            Claims = new List<UserClaim>();
        }
        public string UserId { get; set; }
    }
}
```



```

    public List<UserClaim> Claims { get; set; }
}

```

```

[HttpGet]
0 references
public async Task<IActionResult> ManageUserClaims(string userId)
{
    var user = await userManager.FindByIdAsync(userId);

    if (user == null)
    {
        ViewBag.ErrorMessage = $"User with Id = {userId} cannot be found";
        return View("NotFound");
    }
    // to store
    var existingUserClaims = await userManager.GetClaimsAsync(user);
    // instance of ViewModel class
    var model = new UserClaimsViewModel
    {
        // populate property
        UserId = userId
    };
}

```

list of all claims

method

```

foreach (Claim claim in ClaimsStore.AllClaims)
{
    UserClaim userClaim = new UserClaim
    {
        ClaimType = claim.Type
    };

    // If the user has the claim, set IsSelected property to true, so the checkbox
    // next to the claim is checked on the UI
    if (existingUserClaims.Any(c => c.Type == claim.Type))
    {
        // var storing all claims
        // if the type of the existing claim matches with
        // the type of the claim we are iterating over
        // => true = user has that claim
        userClaim.IsSelected = true;
    }

    // otherwise (default) => false
    // add userClaim object to Claims collection property
    // of view model object
    model.Claims.Add(userClaim);

    return View(model); // display it
}

```

```

[HttpGet]
public async Task<IActionResult> ManageUserClaims(string userId)
{
    var user = await userManager.FindByIdAsync(userId);

    if (user == null)
    {
        ViewBag.ErrorMessage = $"User with Id = {userId} cannot be
found";
        return View("NotFound");
    }
    var existingUserClaims = await userManager.GetClaimsAsync(user);
}

```

```

var model = new UserClaimsViewModel
{
    UserId = userId
};
foreach (Claim claim in ClaimsStore.AllClaims)
{
    UserClaim userClaim = new UserClaim
    {
        ClaimType = claim.Type
    };

    // If the user has the claim, set IsSelected property to true,
    // so the checkbox next to the claim is checked on the UI
    if (existingUserClaims.Any(c => c.Type == claim.Type))
    {
        userClaim.IsSelected = true;
    }

    model.Claims.Add(userClaim);
}

return View(model);
}

```

Administration folder -> Add -> Razor view: ManageUserClaims.cshtml

```

@model UserClaimsViewModel

<form method="post">
    <div class="card">
        <div class="card-header">
            <h2>Manage User Claims</h2>
        </div>
        <div class="card-body">
            @for (int i = 0; i < Model.Claims.Count; i++)
            {
                <div class="form-check m-1">
                    <input type="hidden" asp-for="@Model.Claims[i].ClaimType" />
                    <input asp-for="@Model.Claims[i].IsSelected" class="form-check-input" />
                    <label class="form-check-label" asp-for="@Model.Claims[i].IsSelected">
                        @Model.Claims[i].ClaimType
                    </label>
                </div>
            }
            <div asp-validation-summary="All" class="text-danger"></div>
        </div>
        <div class="card-footer">
            <input type="submit" value="Update" class="btn btn-primary" style="width:auto" />
            <a asp-action="EditUser" asp-route-id="@Model.UserId" class="btn btn-primary" style="width:auto">Cancel</a>
        </div>
    </div>
</form>

```

@model UserClaimsViewModel

```

<form method="post">
    <div class="card">
        <div class="card-header">
            <h2>Manage User Claims</h2>
        </div>
        <div class="card-body">

```

```

        @for (int i = 0; i < Model.Claims.Count; i++)
        {
            <div class="form-check m-1">
                <input type="hidden" asp-for="@Model.Claims[i].ClaimType" />
                <input asp-for="@Model.Claims[i].IsSelected" class="form-
check-input" />
                <label class="form-check-label" asp-
for="@Model.Claims[i].IsSelected">
                    @Model.Claims[i].ClaimType
                </label>
            </div>
        }
        <div asp-validation-summary="All" class="text-danger"></div>
    </div>
    <div class="card-footer">
        <input type="submit" value="Update" class="btn btn-primary"
        style="width:auto" />
        <a asp-action="EditUser" asp-route-id="@Model.UserId"
        class="btn btn-primary" style="width:auto">Cancel</a>
    </div>
</div>
</form>

```

localhost:24077/Administration/ManageUserClaims?userId=

List Create Manage

Manage User Claims

- ☒ Create Role
- ☒ Edit Role
- ☒ Delete Role

Update Cancel